Highlights

Cloud Databases: A Systematic Meta-Survey of Current Challenges, Emerging Paradigms, and Future Research Directions

Author One, Author Two

- First systematic meta-survey synthesizing 58 cloud database surveys published between 2019 and 2025
- Dual-theme taxonomy organizing challenges into 18 cross-cutting topics and 5 lifecycle phases
- Analysis of the LLM–cloud database convergence as an emerging paradigm
- Prioritized research roadmap with short-, medium-, and long-term directions
- Knowledge engineering perspective connecting cloud databases to data and knowledge engineering

Cloud Databases: A Systematic Meta-Survey of Current Challenges, Emerging Paradigms, and Future Research Directions

Author One[a,*], Author Two[a]

[a] *Department of Computer Science, University Name, City, Country*

Abstract

Cloud databases have become the backbone of modern data-driven applications, enabling organizations to store, process, and analyze data at unprecedented scale and elasticity. Over the past seven years (2019–2025), more than 50 survey papers have examined various facets of cloud database technology—spanning cloud-native architectures, hybrid transactional/analytical processing (HTAP), serverless databases, vector databases, AI-for-database optimization (AI4DB), data lakehouses, NoSQL/NewSQL systems, and edge–cloud data management. However, the challenges, open problems, and research directions identified in these surveys remain fragmented across disparate communities and publication venues. This paper presents the first *systematic meta-survey* of cloud database research: a survey of surveys that synthesizes findings from 58 carefully selected survey papers using a rigorous methodology grounded in the Kitchenham–Charters systematic literature review protocol. We organize the scattered landscape of cloud database challenges into a structured, dual-theme taxonomy: (1) *eighteen cross-cutting challenges* that transcend individual system categories, including scalability, consistency trade-offs, security, cost optimization, AI–database convergence, and sustainability; and (2) *five lifecycle-phase challenges* spanning design, deployment, operation, optimization, and evolution of cloud database systems. We further examine the convergence of cloud databases with large language models (LLMs) as an illustrative case study, and propose a priori-

*Corresponding author.

Email addresses: `author.one@university.edu` (Author One), `author.two@university.edu` (Author Two)

tized research roadmap with short-, medium-, and long-term directions. By providing a consolidated, thematically organized view of the field, this meta-survey serves as both a comprehensive reference for researchers entering the cloud database space and a strategic guide for practitioners navigating the rapidly evolving landscape of cloud data management.

Keywords: cloud databases, systematic meta-survey, cloud-native databases, database-as-a-service, distributed databases, data lakehouse, AI4DB, survey of surveys

---

1. Introduction

The cloud database market has experienced explosive growth over the past decade, with global revenues projected to exceed $200 billion by 2028 [1]. This growth reflects a fundamental shift in how organizations manage data: from on-premises, monolithic database servers to elastic, globally distributed, cloud-native data platforms that can scale storage and compute independently on demand [2, 3]. Major cloud providers—Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and Alibaba Cloud—now offer dozens of specialized database services, from relational engines like Amazon Aurora [2] and Azure SQL [4] to purpose-built systems for documents, graphs, time series, key-value pairs, and, most recently, vector embeddings for AI workloads [5].

This rapid evolution has attracted significant research attention. A search across major digital libraries reveals that **more than 50 survey papers** have been published between 2019 and 2025, each examining a particular facet of cloud database technology. These surveys span a wide spectrum of topics: cloud-native database architectures [6], hybrid transactional/analytical processing (HTAP) [7], serverless data management [8], vector database management systems [5], AI-driven database optimization (AI4DB) [9], data lakehouses [10], NoSQL and NewSQL systems [11, 12], edge and fog database systems [13], and many more.

Despite this wealth of individual surveys, the field lacks a *meta-level synthesis*—a comprehensive, systematic effort to consolidate the challenges, open problems, and research directions scattered across these surveys into a unified framework. Each survey naturally focuses on its own domain, leading to a fragmented understanding of the cloud database landscape as a whole. Cross-cutting challenges such as cost optimization, security, consistency trade-offs,

and AI–database convergence are discussed in many surveys but from different angles, without a common organizing framework.

1.1. Contributions and novelty

This paper addresses the gap by presenting the **first systematic meta-survey of cloud database research**—a survey of surveys. Inspired by the methodological approach of Saeed and Omlin [14], who conducted a meta-survey of 73 surveys on Explainable AI (XAI), we apply a rigorous systematic literature review (SLR) protocol based on the Kitchenham–Charters guidelines [15] to identify, evaluate, and synthesize cloud database surveys. Our contributions are as follows:

C1 Systematic identification and classification. We systematically search eight digital libraries and identify 58 survey papers on cloud databases published between 2019 and 2025, classifying them by topic, scope, methodology, and venue.

C2 Dual-theme challenge taxonomy. We organize the scattered challenges into a structured taxonomy with two complementary themes:

- *Theme 1: Eighteen cross-cutting challenges* that recur across multiple survey domains, including scalability, consistency, security, cost modeling, AI–database convergence, and sustainability.
- *Theme 2: Five lifecycle-phase challenges* that map to the stages of designing, deploying, operating, optimizing, and evolving cloud database systems.

C3 LLM–database convergence analysis. We examine the rapidly emerging intersection of large language models (LLMs) and cloud databases as an illustrative case study, covering text-to-SQL, vector databases for retrieval-augmented generation (RAG), and LLM-based database administration.

C4 Research roadmap. We propose a prioritized research roadmap with short-term (1–3 years), medium-term (3–5 years), and long-term (5+ years) directions.

C5 Knowledge engineering perspective. We explicitly connect cloud database challenges to data and knowledge engineering concerns—including

knowledge graphs, ontologies, metadata management, and schema evolution—making this survey particularly relevant to the *Data & Knowledge Engineering* community.

1.2. Research questions

This meta-survey is guided by five research questions, summarized in Table 3 and elaborated in Section 4.

1.3. Paper organization

The remainder of this paper is organized as follows. Section 2 motivates the importance of cloud databases from five complementary perspectives. Section 3 establishes key definitions and distinctions—from cloud-hosted to cloud-native databases, architectural paradigms, data models, and the data platform continuum. Section 4 details our systematic review methodology, including search strategy, selection criteria, quality assessment, and results. Section 5 presents the core contribution: a comprehensive discussion of challenges and research directions organized by cross-cutting themes and lifecycle phases. Section 6 examines the convergence of cloud databases and LLMs. Section 7 proposes a prioritized research roadmap, and Section 8 offers final remarks. Appendix Appendix A provides the complete catalog of surveyed papers.

2. Why Cloud Databases Matter—A Multi-Perspective Motivation

Understanding the significance of cloud databases requires examining the topic from multiple viewpoints. In this section, we motivate the need for a meta-survey by discussing five complementary perspectives, each revealing why the scattered state of survey knowledge is problematic and why a unified synthesis is timely.

2.1. Industry perspective

The global database management system (DBMS) market reached $101 billion in 2023, with cloud deployments accounting for over 65% of total revenue—up from approximately 40% in 2019 [1]. Gartner projects that by 2026, 75% of all databases will be deployed on cloud platforms [1]. This shift is driven by digital transformation initiatives across industries, from financial services adopting cloud databases for real-time fraud detection to

healthcare organizations leveraging cloud data platforms for genomics and precision medicine [16].

The vendor landscape has also diversified significantly. Beyond the hyperscale cloud providers (AWS, Azure, GCP), specialized cloud database vendors such as Snowflake [3], Databricks [17], CockroachDB [18], PlanetScale, and Neon have gained substantial market share. China's cloud database ecosystem—including Alibaba PolarDB [19], TiDB [20], and OceanBase [21]—has matured rapidly. This fragmentation makes it difficult for practitioners to navigate the landscape without a systematic overview.

2.2. Data engineering perspective

Modern data architectures have evolved from simple transactional databases to complex, multi-layered data ecosystems. The emergence of the *data lakehouse* paradigm [17] exemplifies this evolution, combining the flexibility of data lakes with the governance and performance of data warehouses. Systems like Databricks Delta Lake, Apache Iceberg, and Apache Hudi have created a new "open table format" layer that bridges batch and streaming analytics [22].

Simultaneously, the rise of *HTAP* (Hybrid Transactional/Analytical Processing) systems [7] challenges the traditional separation between OLTP and OLAP workloads. Systems such as TiDB [20], SAP HANA [23], and Google AlloyDB promise unified engines that handle both transaction processing and analytical queries. Understanding the trade-offs, maturity, and open challenges of these paradigms requires synthesizing insights across multiple survey papers.

2.3. Knowledge engineering perspective

Cloud databases play a crucial role in knowledge engineering, making this topic directly relevant to the *Data & Knowledge Engineering* (DKE) community. Several connections are worth highlighting:

- **Knowledge graphs in the cloud.** Large-scale knowledge graphs (KGs) such as Google Knowledge Graph, Wikidata, and enterprise KGs are increasingly stored and queried using cloud-hosted graph databases (e.g., Amazon Neptune, Azure Cosmos DB with Gremlin API, Neo4j Aura) [24]. The scalability, consistency, and query optimization challenges of cloud graph databases directly impact KG applications.

- **Ontology-mediated data access.** Cloud environments with heterogeneous data sources benefit from ontology-based data integration (OBDI) techniques [25], where ontologies provide a unified conceptual layer over distributed cloud databases.

- **Metadata and schema management.** Cloud data catalogs and metadata management systems (e.g., AWS Glue Data Catalog, Apache Atlas) implement knowledge engineering principles for organizing, discovering, and governing data assets [26].

- **AI–knowledge engineering convergence.** The integration of machine learning with database systems (AI4DB) [9] draws on knowledge engineering concepts such as learned models for query optimization, workload characterization, and automated tuning.

2.4. Economic and cost perspective

Cloud databases introduce novel economic dynamics compared to on-premises deployments. The shift from capital expenditure (CapEx) to operational expenditure (OpEx) models, combined with pay-per-use pricing and complex multi-dimensional billing (compute, storage, I/O, network egress, backups), creates significant challenges in cost prediction and optimization [27].

Research has shown that cloud database costs can vary by orders of magnitude depending on configuration choices, query patterns, and pricing model selection [27]. The total cost of ownership (TCO) analysis becomes particularly complex for multi-cloud or hybrid cloud deployments. Several surveys touch on cost-related challenges [6, 8], but a consolidated analysis is lacking.

2.5. Regulatory and compliance perspective

Data sovereignty regulations such as the European Union's General Data Protection Regulation (GDPR) [28], China's Data Security Law, and sector-specific requirements (e.g., HIPAA for healthcare, PCI-DSS for payment data) impose strict constraints on how and where cloud databases store and process data [29].

Multi-region deployment, data residency requirements, cross-border data transfer restrictions, and audit trail obligations all impact cloud database architecture and operations. The compliance landscape interacts with technical challenges such as geo-distributed consistency, encryption, access control, and data lineage—topics discussed in various surveys but rarely addressed in an integrated manner.

Table 1: Comparison of cloud database deployment models.

Characteristic	Cloud-Hosted	Cloud-Native	Cloud-Born
Definition	Traditional DBMS deployed on cloud VMs	DBMS redesigned to exploit cloud infrastructure	DBMS designed from scratch for the cloud
Architecture	Monolithic, tightly coupled	Disaggregated compute/storage	Fully disaggregated, microservice-based
Elasticity	Manual scaling (vertical)	Automatic scaling (horizontal)	Instant, fine-grained scaling
Examples	MySQL on EC2, Oracle on Azure VM	Amazon Aurora [2], Azure SQL [4]	Snowflake [3], Neon, CockroachDB [18]
State sharing	Shared-nothing (typically)	Shared-storage	Disaggregated, shared cloud services
Multi-tenancy	VM-level isolation	Engine-level isolation	Native multi-tenant design

3. From Cloud Databases to Cloud-Native Data Platforms

Before analyzing the surveyed literature, we establish key concepts, definitions, and distinctions that form the vocabulary of cloud database research. This section parallels the definitional groundwork laid by Saeed and Omlin [14] for XAI terminology.

3.1. Cloud-hosted vs. cloud-native vs. cloud-born databases

The term "cloud database" encompasses a spectrum of architectures with fundamentally different design philosophies. We distinguish three categories, compared in Table 1:

Cloud-hosted databases are traditional database engines (e.g., PostgreSQL, MySQL, Oracle) deployed on cloud virtual machines. They benefit from cloud infrastructure (networking, storage volumes, automated backups) but do not exploit cloud-native primitives such as elastic compute pools or disaggregated storage [6].

Cloud-native databases are existing or new engines that have been substantially re-architected to leverage cloud services. Amazon Aurora [2] exemplifies this approach: it re-implements the storage layer of MySQL/PostgreSQL to use a distributed, log-structured storage service while keeping the SQL engine largely intact. Microsoft's Socrates [4] takes SQL Server and disaggregates its components across cloud microservices.

Cloud-born databases are designed entirely from the ground up for cloud deployment, with no legacy on-premises heritage. Snowflake [3] pioneered this model for data warehousing, while CockroachDB [18] and Neon represent cloud-born approaches for transactional and PostgreSQL-compatible workloads, respectively.

3.2. Database-as-a-Service (DBaaS) spectrum

Database-as-a-Service (DBaaS) represents the fully managed end of the cloud database spectrum, where the cloud provider handles provisioning, patching, scaling, backup, and high availability [30]. The DBaaS model has evolved along several dimensions:

- **Managed databases**: Provider-managed instances of open-source or commercial engines (e.g., Amazon RDS, Azure Database for PostgreSQL).
- **Serverless databases**: Fully elastic databases that scale to zero when idle and automatically provision resources based on demand (e.g., Amazon Aurora Serverless, Azure Cosmos DB serverless, Neon) [8].
- **Autonomous databases**: Self-driving databases that use ML to automate tuning, indexing, partitioning, and anomaly detection (e.g., Oracle Autonomous Database) [31].

3.3. Architectural paradigms

Cloud database architectures have evolved through three major paradigms, illustrated in Figure 1:

Shared-nothing architectures partition data across independent nodes, each with its own CPU, memory, and disk. Traditional distributed databases (e.g., early Spanner [32], Cassandra) follow this model. While horizontally scalable, resharding and rebalancing are expensive.

Shared-disk (cloud storage) architectures decouple compute from storage, allowing compute nodes to share a common cloud storage layer. Amazon Aurora [2] and Snowflake [3] exemplify this paradigm, enabling independent scaling of compute and storage.

Fully disaggregated architectures further decompose the database engine into independently scalable microservices—query processing, transaction management, logging, metadata management, and storage [4]. Microsoft Socrates and emerging systems like PolarDB [19] pursue this vision.

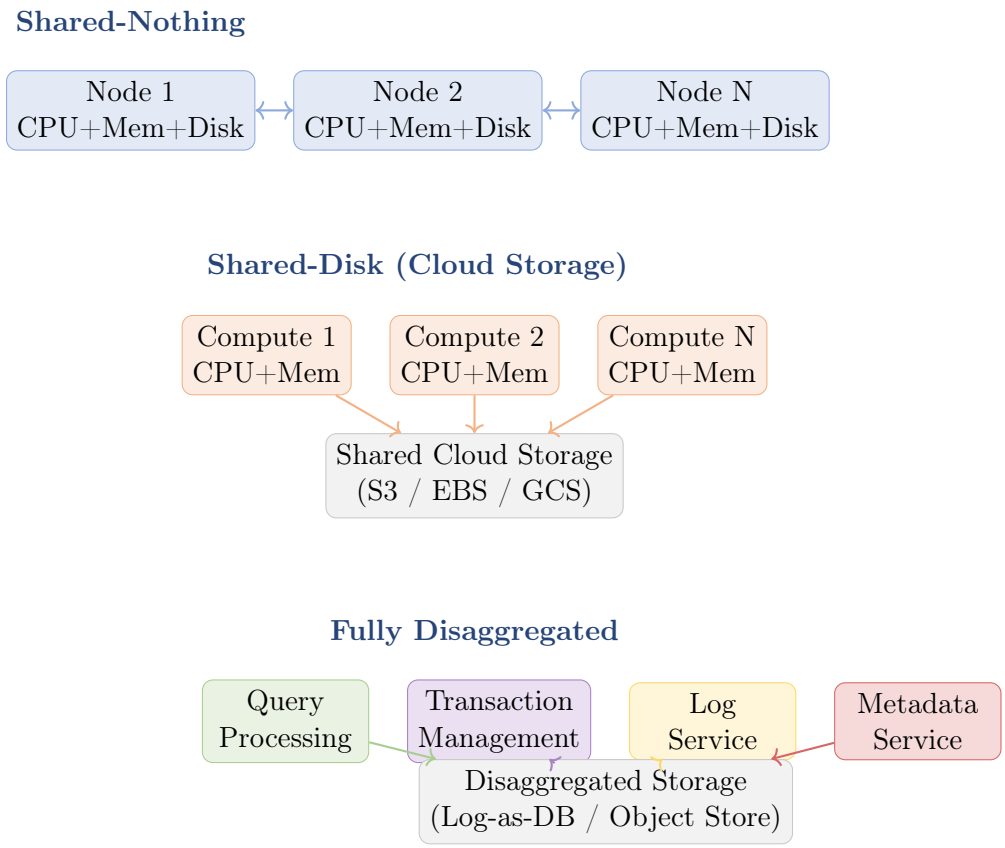

Figure 1: Evolution of cloud database architectures: from shared-nothing clusters to shared-disk (cloud storage) to fully disaggregated microservice architectures.

3.4. Data models in the cloud

Cloud platforms support a diverse ecosystem of data models, each optimized for different workload characteristics. Table 2 compares the major data models available in cloud database services.

The trend toward *multi-model databases* [33] reflects the desire to avoid managing separate database systems for different data models. Azure Cosmos DB, for instance, supports document, graph, key-value, column-family, and table APIs through a single backend. However, multi-model integration introduces challenges in query optimization, consistency semantics, and performance trade-offs [33].

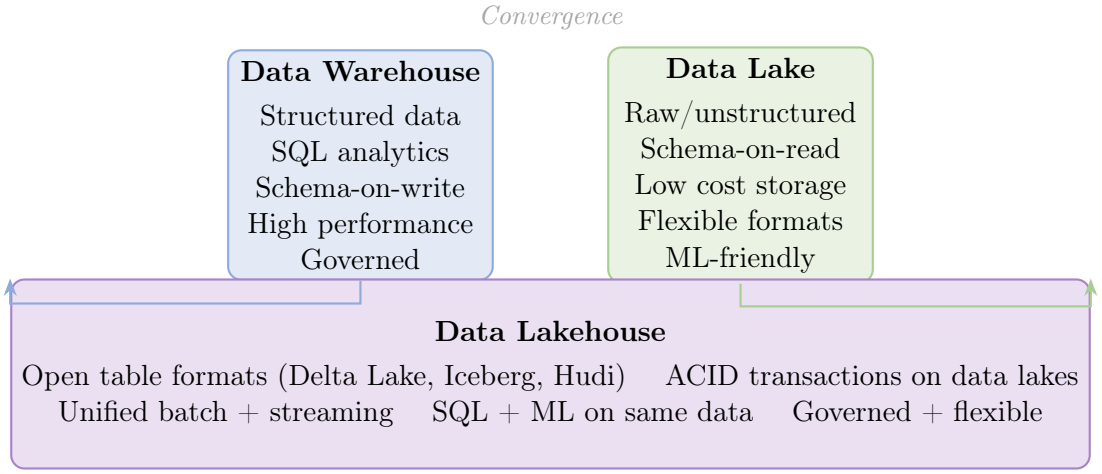

Figure 2: The data platform continuum: data warehouses and data lakes converge into the data lakehouse paradigm.

3.5. The data platform continuum: warehouses, lakes, and lakehouses

The cloud data management landscape has converged around three complementary paradigms, illustrated in Figure 2:

Data warehouses (e.g., Snowflake [3], BigQuery [34], Redshift) provide high-performance SQL analytics over structured, curated data with strong governance and schema enforcement.

Data lakes (e.g., Amazon S3 + Athena, Azure Data Lake Storage) offer cost-effective storage for raw, semi-structured, and unstructured data, enabling schema-on-read flexibility for data science and ML workloads.

Data lakehouses [17] combine the best of both paradigms by adding ACID transactions, schema enforcement, and data governance to open data lake storage through open table formats such as Delta Lake, Apache Iceberg, and Apache Hudi [22].

4. Systematic Review Planning and Execution

We follow the systematic literature review (SLR) guidelines established by Kitchenham and Charters [15], which have been widely adopted in software engineering and computer science. Our methodology mirrors the approach used by Saeed and Omlin [14] for their XAI meta-survey, adapted for the cloud database domain.

4.1. Research questions

Our meta-survey is guided by five research questions, presented in Table 3.

4.2. Search strategy

We searched eight major digital libraries and indexing services, as detailed in Table 4.

The search string was constructed by combining cloud database terminology with survey-indicating terms:

```
("cloud database" OR "cloud-native database" OR "DBaaS" OR
"database-as-a-service" OR "serverless database" OR "HTAP" OR
"NewSQL" OR "data lakehouse" OR "vector database" OR "AI4DB"
OR "learned index" OR "NoSQL" OR "distributed database" OR
"multi-model database" OR "graph database" OR "edge database"
OR "fog database" OR "autonomous database")
AND
("survey" OR "review" OR "tutorial" OR "taxonomy" OR "systematic"
OR "state-of-the-art" OR "overview")
```

4.3. Study selection

We applied the inclusion and exclusion criteria listed in Table 5 in a two-phase process: (1) title and abstract screening, and (2) full-text assessment.

4.4. Quality assessment

Each candidate survey was evaluated against eight quality assessment criteria (QA1–QA8), scored on a 3-point scale (0 = No, 0.5 = Partially, 1 = Yes). Papers scoring below 4 out of 8 were excluded.

4.5. Results: study selection and demographics

Figure 3 presents the PRISMA flow diagram of our selection process. From an initial set of 412 candidate papers, we selected 58 surveys for inclusion in the meta-survey after applying all criteria.

Figure 4 shows the distribution of selected surveys by publication year, revealing a clear upward trend from 2019 to 2024.

Figure 5 shows the distribution by primary topic domain. Cloud-native architectures, distributed/NoSQL systems, and AI4DB are the most frequently surveyed topics.

Figure 6 shows the distribution by venue type. Journal publications account for the majority, followed by conference proceedings and arXiv preprints.

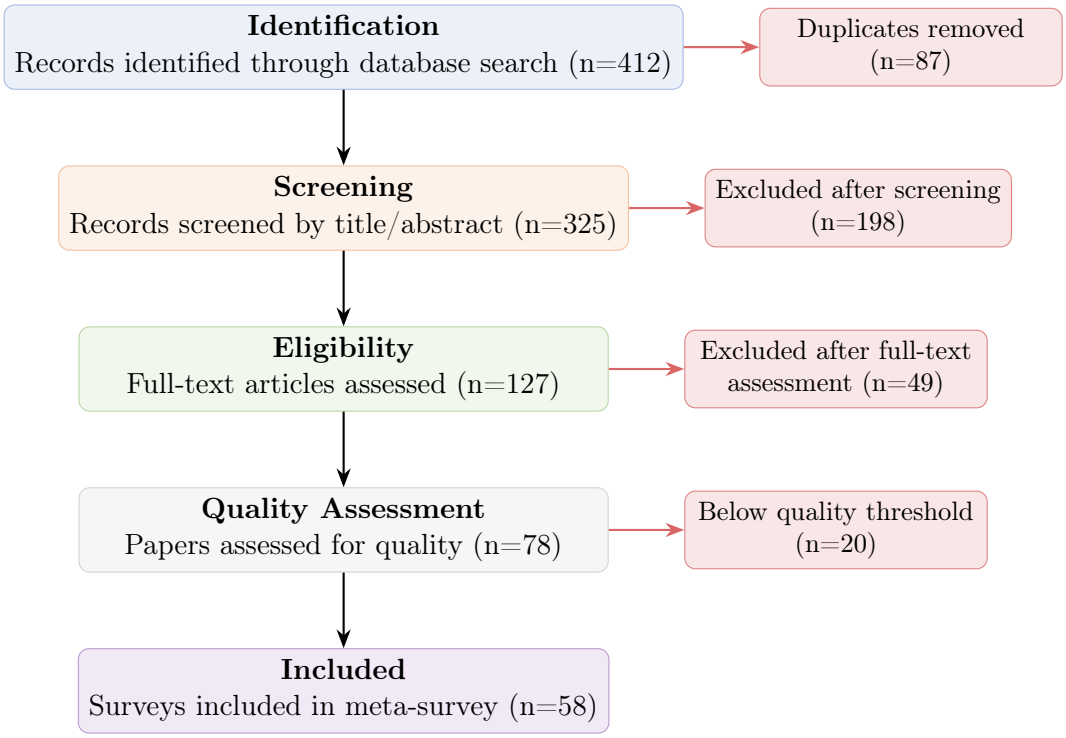

Figure 3: PRISMA flow diagram of the study selection process.

5. Discussion—Challenges and Research Directions

This section presents the core contribution of our meta-survey: a comprehensive, structured analysis of the challenges and research directions identified across the 58 surveyed papers. Following the dual-theme approach of Saeed and Omlin [14], we organize our discussion into two complementary themes:

1. **Theme 1: Cross-cutting challenges** (Section 5.1)—eighteen recurring challenges that transcend individual system categories.
2. **Theme 2: Lifecycle-phase challenges** (Section 5.2)—challenges organized by the five phases of the cloud database lifecycle.

Figure 7 provides a high-level overview of our dual-theme taxonomy.

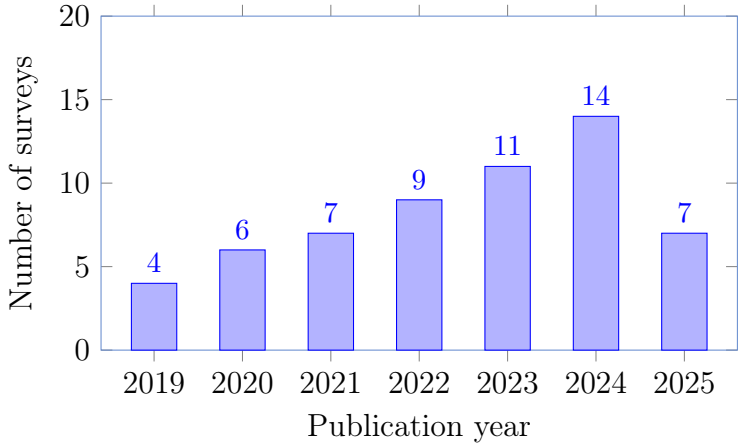

Figure 4: Distribution of the 58 selected surveys by publication year (2019–2025).

5.1. Theme 1: Cross-cutting challenges

We identify eighteen cross-cutting challenges that recur across multiple cloud database survey domains. For each challenge, we present: (1) the definition and context, (2) the current state as reported in surveyed papers, (3) open problems, and (4) research directions with DKE relevance where applicable.

5.1.1. Scalability and elasticity

Context. Scalability—the ability to handle growing workloads by adding resources—and elasticity—the ability to dynamically provision and release resources in response to demand fluctuations—are foundational requirements for cloud databases [6, 11].

Current state. Cloud-native databases like Aurora [2] and Snowflake [3] achieve storage elasticity through disaggregation, while compute elasticity is provided by auto-scaling mechanisms. Serverless databases [8] push elasticity further by scaling to zero. However, surveys report that true fine-grained elasticity—scaling individual components (e.g., a single query operator) rather than entire instances—remains an open challenge [6].

Open problems.

- Cold-start latency in serverless databases undermines real-time elasticity [8].

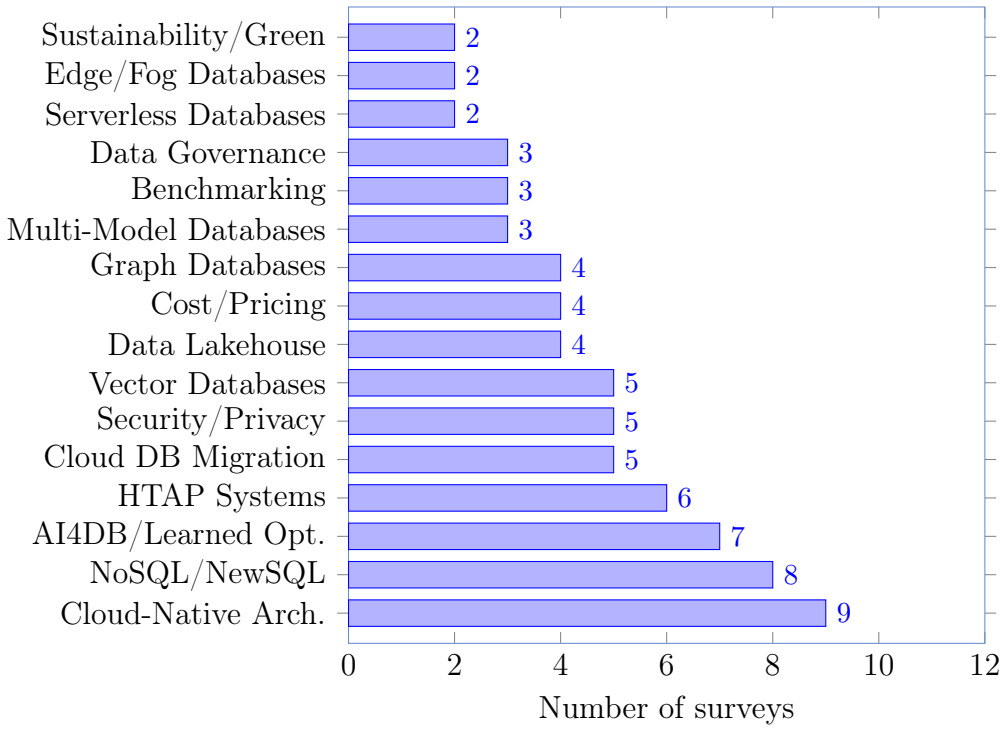

Figure 5: Distribution of the 58 selected surveys by primary topic domain.

- State management during scale-out/scale-in operations can cause transient performance degradation.
- Predictive autoscaling—anticipating workload spikes before they occur—requires accurate workload forecasting models [35].
- Cross-component elasticity in disaggregated architectures (independently scaling query processing, transaction management, and storage) is poorly understood.

Research directions. Developing formal models for *elastic data architectures*—capturing the relationships between workload patterns, resource allocation, performance guarantees, and cost—is a knowledge engineering challenge that could benefit from ontological modeling of cloud database configurations.

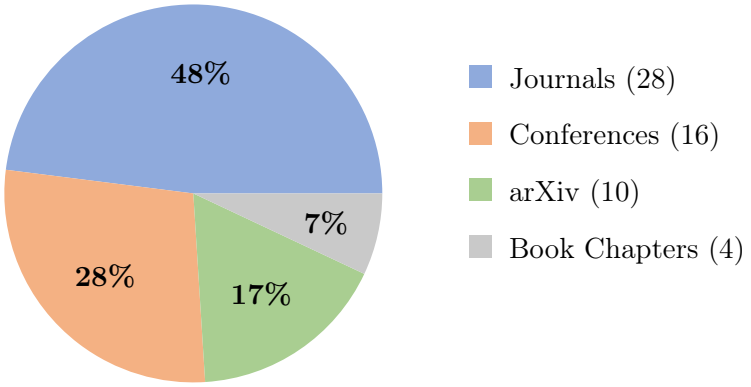

Figure 6: Distribution of the 58 selected surveys by venue type.

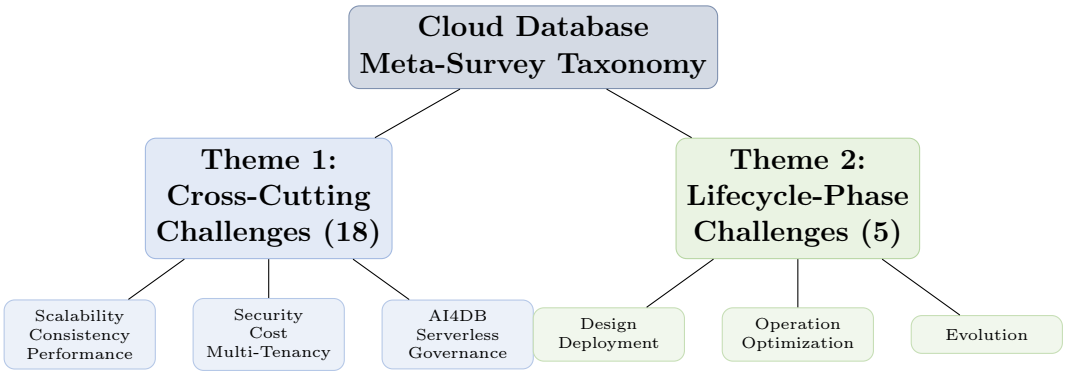

Figure 7: High-level taxonomy of the cloud database meta-survey, organized into two complementary themes.

5.1.2. Consistency, availability, and partition tolerance revisited

Context. The CAP theorem [36] and its refinement PACELC [37] have long shaped distributed database design. In the cloud era, these trade-offs take on new dimensions due to geo-distribution, multi-region deployment, and the diversity of consistency models offered by cloud databases [38].

Current state. Cloud databases offer a spectrum of consistency levels. Google Spanner [32] provides external consistency (linearizability) using TrueTime, while Cosmos DB offers five consistency levels from strong to eventual. Surveys on distributed databases [11, 12] and HTAP systems [7] consistently identify consistency trade-offs as a central challenge.

Open problems.

- Developers struggle to reason about the implications of different consistency levels for application correctness [38].
- Adaptive consistency—dynamically adjusting consistency levels based on workload requirements—is proposed but rarely implemented in production systems.
- Cross-model consistency in multi-model databases (e.g., ensuring consistency between document and graph views of the same data) is underexplored [33].

Research directions. Formal verification and knowledge-based reasoning about consistency guarantees, including ontology-based specifications of consistency contracts, could help bridge the gap between theoretical models and practical developer needs.

5.1.3. Performance optimization and benchmarking

Context. Performance optimization in cloud databases spans query optimization, indexing, caching, resource allocation, and workload management. Benchmarking provides the empirical foundation for evaluating and comparing systems [39].

Current state. Surveys on AI4DB [9] and cloud-native databases [6] highlight significant progress in learned query optimization [40], learned indexes [41], and automated knob tuning [42]. However, cloud-specific factors—network latency, storage disaggregation overhead, multi-tenant interference, and cold-start effects—complicate traditional optimization approaches. Existing benchmarks (TPC-C, TPC-H, YCSB) are criticized for not capturing cloud-specific characteristics [39].

Open problems.

- Cloud-specific benchmarks that account for elasticity, multi-tenancy, pricing, and variable network performance are lacking.
- Fair cross-system comparison is difficult due to differing cloud provider pricing, hardware, and service-level agreements.
- End-to-end performance in disaggregated architectures involves complex interactions between independently scaled components.

Research directions. A *benchmark knowledge base* that formalizes benchmark configurations, workload characteristics, hardware specifications, and results using structured knowledge representations would enable reproducible and comparable cloud database evaluations.

5.1.4. Security and privacy

Context. Cloud databases introduce unique security challenges compared to on-premises systems: shared infrastructure, network-based access, provider trust, and the expanded attack surface of distributed systems [43].

Current state. Surveys on cloud database security [43, 44] cover encryption at rest and in transit, access control, audit logging, and vulnerability management. Advanced topics include homomorphic encryption for query processing on encrypted data [45], secure multi-party computation, and confidential computing (e.g., Intel SGX enclaves for database operations).

Open problems.

- Performance overhead of encryption, especially for analytical workloads, remains prohibitive.
- Key management across multi-cloud and hybrid deployments is complex and error-prone.
- Insider threat models for cloud providers (the "honest but curious" provider) are difficult to mitigate without significant performance penalties.
- Privacy-preserving analytics that comply with GDPR's right to erasure while maintaining data utility.

Research directions. *Privacy ontologies* that formally model data sensitivity, access policies, and regulatory requirements could enable automated compliance checking and policy enforcement across cloud database deployments.

5.1.5. Cost modeling and optimization

Context. Cloud databases employ complex pricing models combining compute hours, storage capacity, I/O operations, network transfer, backup storage, and premium features (encryption, audit logs, multi-region replication) [27].

Current state. Recent studies [27] demonstrate that cloud database costs can vary by 10–100× depending on configuration choices. Surveys on serverless databases [8] and cloud-native systems [6] identify cost as a critical concern but offer limited quantitative analysis.

Open problems.

- Cost estimation for complex analytical queries before execution is unreliable.
- Multi-objective optimization (cost vs. performance vs. consistency) lacks mature frameworks.
- Reserved vs. on-demand vs. spot instance strategies for database workloads are poorly studied.
- Vendor lock-in makes cost comparison across providers difficult.

Research directions. Developing formal *cost knowledge models*—ontologies capturing pricing structures, workload profiles, and optimization strategies—could enable intelligent, automated cost optimization across cloud database services.

5.1.6. Multi-tenancy and resource isolation

Context. Multi-tenancy allows cloud providers to serve multiple customers from shared infrastructure, reducing costs and improving resource utilization. However, it introduces challenges in performance isolation, data security, and fair resource allocation [6].

Current state. Cloud-native databases implement multi-tenancy at different levels: shared-nothing (dedicated instances per tenant), shared-process (multiple tenants in one engine with logical isolation), and shared-table (tenants share physical tables with row-level security). Surveys report that noisy neighbor effects—where one tenant's workload degrades another's performance—remain a significant operational challenge [46].

Open problems.

- Achieving strong performance isolation without dedicated hardware is difficult.
- Workload-aware resource scheduling that adapts to heterogeneous tenant workloads.
- Balancing isolation guarantees with resource efficiency and cost.

5.1.7. Data model diversity and multi-model integration

Context. The proliferation of specialized data models (relational, document, graph, key-value, wide-column, vector, time-series) in cloud environments creates challenges for applications that need to query across multiple models [33].

Current state. Multi-model databases like Azure Cosmos DB, ArangoDB, and SurrealDB offer multiple data model interfaces over a single backend. Surveys on NoSQL [11] and multi-model systems [33] report growing adoption but highlight limitations in cross-model query optimization and unified transaction semantics.

Open problems.

- Unified query languages that span relational, graph, document, and vector operations.
- Cross-model query optimization—optimizing queries that combine, e.g., graph traversals with relational joins and vector similarity search.
- Schema mapping and data transformation between models in heterogeneous environments.

Research directions. *Knowledge representation in multi-model environments*—using ontologies to provide a unified conceptual layer over diverse data models—is a direct DKE opportunity.

5.1.8. Query processing across heterogeneous engines

Context. Modern cloud data architectures often involve multiple specialized database engines (OLTP, OLAP, graph, search, vector) that must be queried in concert. Federated query processing and data virtualization are key enablers [7].

Current state. Systems like Presto/Trino, BigQuery Omni, and AWS Lake Formation provide federated query capabilities. HTAP surveys [7] discuss the challenges of routing queries to appropriate engines. However, cross-engine optimization remains primitive—typically limited to simple predicate pushdown.

Open problems.

- Cost-based optimization across heterogeneous engines with different cost models and capabilities.

- Maintaining transactional guarantees across engine boundaries.
- Semantic understanding of data across engines to enable intelligent query routing.

Research directions. *Ontology-mediated query answering (OMQA)* [25] can provide a principled framework for querying heterogeneous cloud database engines through a unified conceptual schema.

5.1.9. Storage disaggregation and log-as-database

Context. Storage disaggregation—separating compute from storage into independently scalable layers—is a defining architectural trend in cloud databases. The "log-as-database" pattern, pioneered by Aurora [2], pushes intelligence into the storage layer [6].

Current state. Aurora's innovation of making the redo log the "database" reduced write amplification and enabled fast failover. Systems like PolarDB [19] and Socrates [4] further disaggregate components. Surveys [6] report that disaggregation simplifies operations but introduces challenges in remote I/O latency, cache coherence, and log management.

Open problems.

- Minimizing the performance impact of remote storage access (network vs. local SSD latency gap).
- Cache management across compute and storage layers in disaggregated architectures.
- Log management at scale—compaction, garbage collection, and efficient replay.

5.1.10. Transaction processing in distributed cloud environments

Context. Distributed transaction processing in cloud databases must balance ACID guarantees, performance, and scalability. The design space ranges from strongly consistent protocols (2PC, Paxos/Raft-based) to relaxed models (eventual consistency, CRDTs) [32, 18].

Current state. Google Spanner [32] demonstrated that globally distributed strongly consistent transactions are feasible, using synchronized clocks (TrueTime). CockroachDB [18] and YugabyteDB provide similar guarantees using hybrid logical clocks. NewSQL surveys and HTAP surveys [7]

report that combining strong consistency with high throughput for mixed workloads remains challenging.

Open problems.

- Reducing commit latency for geo-distributed transactions without sacrificing consistency.
- Efficient conflict resolution for concurrent transactions in multi-region deployments.
- Transaction processing for mixed HTAP workloads without mutual interference.

5.1.11. Fault tolerance, recovery, and high availability

Context. Cloud databases must tolerate hardware failures, network partitions, availability zone outages, and even entire region failures while maintaining data durability and minimizing downtime [2, 6].

Current state. Modern cloud databases achieve high availability through multi-AZ replication (typically 3-way) with automated failover. Aurora [2] uses a quorum-based protocol across 6 replicas in 3 AZs. Surveys report that recovery time objectives (RTO) of seconds and recovery point objectives (RPO) of zero are achievable for single-region deployments but remain challenging for multi-region configurations.

Open problems.

- Multi-region disaster recovery with near-zero RPO and RTO.
- Consistent recovery in disaggregated architectures where state is spread across components.
- Chaos engineering and fault injection frameworks specific to cloud databases.

5.1.12. Data migration and cross-cloud portability

Context. Organizations frequently need to migrate databases between on-premises and cloud, between cloud providers, or between different cloud database services. Migration surveys [47] highlight the complexity of maintaining data consistency, minimizing downtime, and handling schema transformations during migration.

Current state. Cloud providers offer migration services (AWS DMS, Azure Database Migration Service, GCP Database Migration Service), but migrations remain error-prone and labor-intensive. Schema and data model differences between source and target systems are a major pain point. Vendor lock-in—through proprietary APIs, data formats, and features—further complicates portability [48].

Open problems.

- Zero-downtime migration for large databases with continuous write workloads.
- Automated schema mapping and transformation between heterogeneous systems.
- Standardized portability layers that abstract provider-specific features.

Research directions. *Knowledge-based schema mapping*—using ontological representations of schema semantics to automate migration—connects directly to DKE's core strengths in schema integration and data exchange.

5.1.13. Edge–cloud data continuum

Context. The proliferation of IoT devices, autonomous vehicles, and smart city infrastructure has created demand for database capabilities at the network edge, working in concert with cloud databases [13].

Current state. Edge and fog database surveys [13] identify architectures for distributing data processing between edge devices, fog nodes, and cloud backends. Systems like Azure SQL Edge, AWS Outposts, and CockroachDB's locality-aware features address parts of this challenge.

Open problems.

- Data consistency across the edge–fog–cloud continuum with intermittent connectivity.
- Intelligent data placement—deciding which data to cache, replicate, or move between edge and cloud.
- Resource-constrained query processing on edge devices with limited compute and memory.
- Privacy-preserving data aggregation at the edge before cloud upload.

5.1.14. AI–database convergence (AI4DB and DB4AI)

Context. The convergence of AI and databases manifests in two complementary directions: AI4DB (using AI to optimize database systems) and DB4AI (using database techniques to support AI/ML workloads) [9].

Current state. AI4DB surveys [9] document progress in learned query optimization [40], learned indexes [41], automated knob tuning [42], learned cardinality estimation, and workload forecasting [35]. DB4AI encompasses in-database ML (e.g., Google BigQuery ML, Amazon Redshift ML) and optimized data serving for training pipelines.

Open problems.

- Deploying learned components in production cloud databases with safety guarantees (graceful degradation when ML models fail).
- Training data management for learned database components—how to collect, curate, and update training data from production workloads.
- Transfer learning across database instances, workloads, and hardware configurations.
- Explainability of AI-driven database decisions (why did the optimizer choose this plan?).

Research directions. *Knowledge engineering for self-driving databases*—integrating domain knowledge about database internals with learned models—represents a promising approach to building more robust and explainable autonomous database systems.

5.1.15. Serverless and autonomous database operations

Context. Serverless databases abstract away all infrastructure management, automatically provisioning resources, scaling, patching, and tuning without human intervention. Autonomous databases go further by using ML to self-diagnose and self-heal [31, 8].

Current state. Aurora Serverless v2, Azure Cosmos DB serverless tier, Neon, and PlanetScale offer serverless interfaces. Oracle Autonomous Database represents the most ambitious autonomous database initiative. Surveys [8] report that serverless models are well-suited for variable workloads but face challenges with sustained high-throughput workloads and cold-start latency.

Open problems.

- Achieving predictable performance under serverless autoscaling.
- Cost–performance trade-off transparency in serverless models.
- Full-stack autonomy: integrating self-tuning, self-healing, and self-securing capabilities.

5.1.16. Data governance, provenance, and lineage

Context. As cloud data ecosystems grow in complexity, tracking data origin (provenance), transformation history (lineage), and enforcing governance policies becomes critical for compliance, debugging, and trust [26].

Current state. Data governance surveys identify the need for metadata management, data cataloging, access control, data quality monitoring, and regulatory compliance. Cloud-native governance tools (AWS Glue Data Catalog, Azure Purview, Google Dataplex) provide partial solutions, but surveys report fragmentation and lack of standardization.

Open problems.

- End-to-end data lineage across heterogeneous cloud services and processing engines.
- Automated compliance checking against evolving regulatory requirements.
- Provenance-aware query processing that tracks how results are derived.

Research directions. *Knowledge graphs for metadata management*—using KG technology to represent, link, and reason about metadata across cloud data assets—is a natural DKE contribution.

5.1.17. Sustainability and energy efficiency

Context. Data centers consumed an estimated 1–1.5% of global electricity in 2023, with projections of 3–4% by 2030 due to AI and cloud growth [49]. Cloud databases, as heavy consumers of compute and storage resources, have a direct environmental impact.

Current state. Green computing surveys identify opportunities for energy-efficient query processing, workload scheduling to exploit renewable energy availability, and carbon-aware data placement. Major cloud providers now report carbon footprint metrics (AWS Customer Carbon Footprint Tool,

Google Cloud Carbon Footprint), but database-level energy optimization is nascent.

Open problems.

- Energy-proportional database systems that consume resources proportional to actual workload.
- Carbon-aware data placement and replication—favoring regions with cleaner energy grids.
- Benchmarking frameworks that include energy and carbon metrics alongside performance.

5.1.18. Standardization and interoperability

Context. The proliferation of cloud database systems, query languages, APIs, and data formats creates an interoperability crisis. While SQL remains the lingua franca for relational databases, graph databases alone use multiple query languages (Cypher, SPARQL, Gremlin, GQL), and NoSQL systems have proprietary APIs [24].

Current state. The emergence of GQL (Graph Query Language) as an ISO standard, the SQL/PGQ extension for property graphs in SQL, and open table formats (Iceberg, Delta Lake, Hudi) represent progress toward standardization. However, surveys report that cloud vendors continue to differentiate through proprietary extensions, making true interoperability elusive.

Open problems.

- Universal query interfaces that span relational, graph, document, and vector operations.
- Standardized cloud database APIs for provisioning, monitoring, and management.
- Open data formats and protocols to reduce vendor lock-in.

5.2. Theme 2: Challenges by cloud database lifecycle phases

Complementing the cross-cutting perspective, we organize challenges by the five phases of the cloud database lifecycle: design, deployment, operation, optimization, and evolution. Figure 8 illustrates this lifecycle with key challenges mapped to each phase.

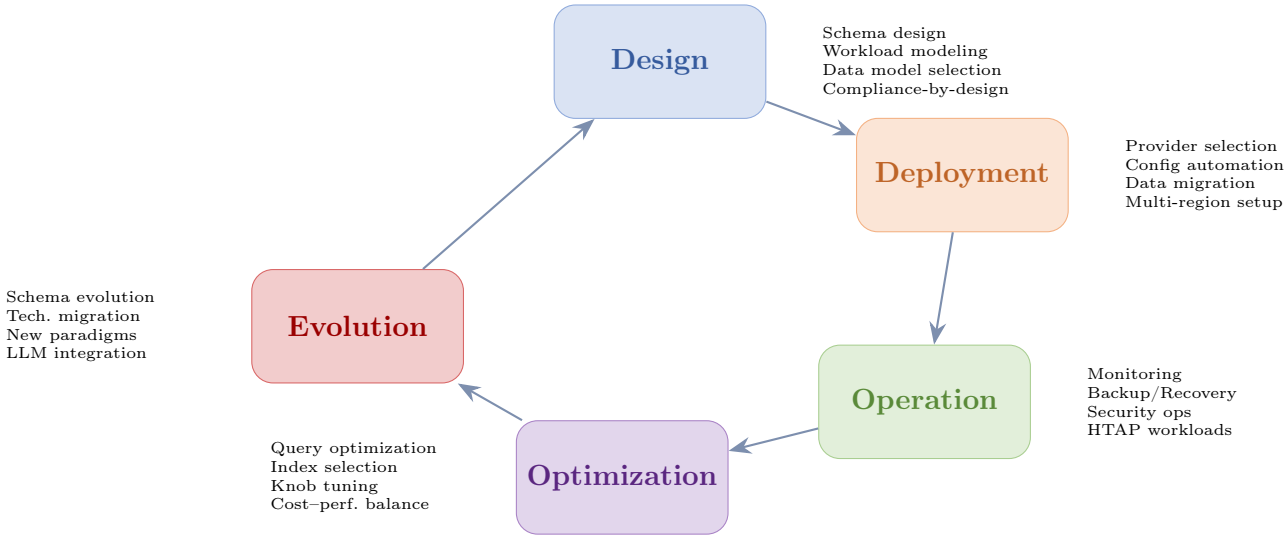

Figure 8: The cloud database lifecycle with key challenges mapped to each phase.

5.2.1. Design phase

The design phase encompasses decisions made before a cloud database is deployed, including schema design, data model selection, workload characterization, and compliance architecture.

Schema design for cloud-native environments. Traditional schema design principles (normalization, ER modeling) must be adapted for cloud databases where denormalization may improve performance due to the cost of distributed joins [6]. Document and wide-column stores require fundamentally different schema design approaches centered on access patterns rather than entity relationships [11].

Workload characterization. Accurate workload modeling is essential for selecting the right cloud database service, configuring resources, and estimating costs. Surveys on AI4DB [9] discuss workload forecasting using ML, but applying these techniques during the design phase—before production data exists—remains challenging.

Data model selection. Choosing between relational, document, graph, key-value, wide-column, vector, or multi-model databases requires understanding workload characteristics, query patterns, consistency requirements, and cost constraints. No systematic decision framework exists that accounts for all these dimensions [33].

Compliance-by-design. Incorporating regulatory requirements (GDPR, HIPAA, data sovereignty) into the initial database design rather than retrofitting compliance is increasingly recognized as essential [29]. This includes data residency constraints, encryption requirements, and audit trail design.

5.2.2. Deployment phase

The deployment phase covers provider selection, infrastructure configuration, data ingestion, and multi-region setup.

Provider selection and vendor lock-in. Choosing a cloud database provider involves evaluating technical capabilities, pricing, compliance certifications, geographic coverage, and ecosystem integration. Vendor lock-in—through proprietary features, APIs, and data formats—is consistently identified as a top concern across surveys [48, 6].

Configuration automation. Infrastructure-as-Code (IaC) tools (Terraform, Pulumi, CloudFormation) enable automated database provisioning, but optimal configuration selection—instance types, storage tiers, replication settings, parameter tuning—often requires deep expertise that is difficult to automate [42].

Data ingestion and migration. Initial data loading and ongoing ingestion from diverse sources (operational databases, streaming platforms, file systems) into cloud databases involves ETL/ELT pipeline design, schema mapping, and data quality assurance [47].

Multi-region deployment. Deploying databases across multiple geographic regions for latency, availability, and compliance reasons introduces challenges in data synchronization, conflict resolution, and region-aware routing [32].

5.2.3. Operation phase

The operation phase covers the day-to-day management of running cloud database systems.

Monitoring and observability. Cloud databases generate vast amounts of telemetry data (metrics, logs, traces). Effective monitoring requires understanding normal behavior patterns, detecting anomalies, and correlating events across distributed components [35]. Surveys on autonomous databases [31] emphasize the need for ML-driven anomaly detection and root cause analysis.

Backup, recovery, and disaster recovery. While cloud providers automate basic backups (point-in-time recovery, snapshots), disaster recovery

across regions, testing recovery procedures, and meeting stringent RPO/RTO targets remain operational challenges [2].

Security operations. Ongoing security management includes access control auditing, encryption key rotation, vulnerability patching, network security configuration, and compliance monitoring. The dynamic nature of cloud environments (auto-scaling, serverless) complicates security posture management [44].

Real-time workload management (HTAP). For HTAP systems [7], operational challenges include managing resource contention between transactional and analytical workloads, maintaining freshness guarantees for analytical queries, and preventing analytical queries from degrading OLTP performance.

5.2.4. Optimization phase

The optimization phase encompasses techniques for improving performance and efficiency of running cloud databases.

Learned query optimization. ML-based query optimizers [40, 9] that learn from workload history to generate better query plans represent a paradigm shift from traditional cost-based optimization. However, deploying learned optimizers in production requires addressing safety (avoiding catastrophic regressions), training efficiency, and adaptability to workload changes.

Automated index and materialized view selection. Cloud databases increasingly offer automated index recommendation (e.g., Azure SQL automatic tuning, Amazon Redshift automatic materialized views), but surveys [9] report that these systems are often conservative, covering only a fraction of possible optimizations.

Knob tuning. Database configuration parameters (buffer pool size, parallelism degree, checkpoint intervals, etc.) significantly impact performance. ML-based tuning systems [42] can explore the high-dimensional configuration space more effectively than manual tuning, but transferring tuning knowledge across different workloads and hardware remains an open problem.

Cost–performance trade-off optimization. In cloud environments, performance optimization cannot be divorced from cost. The optimal configuration minimizes cost subject to performance constraints (or maximizes performance subject to budget constraints), requiring multi-objective optimization [27].

5.2.5. Evolution phase

The evolution phase addresses long-term changes in database systems, technologies, and requirements.

Schema evolution and online DDL. Changing database schemas in production without downtime (online DDL) is a critical capability for agile development. Cloud-native databases have improved online DDL support, but complex migrations (column type changes, table splits, data backfills) remain challenging [6].

Technology migration. Migrating from one database technology to another (e.g., from a relational database to a document store, or from a legacy system to a cloud-native database) is a major undertaking that surveys identify as poorly supported by existing tools [47].

Adapting to new paradigms. The emergence of vector databases for AI workloads [5], the lakehouse paradigm [17], and serverless computing [8] requires existing systems to evolve or be replaced. Understanding when and how to adopt new paradigms is a strategic challenge.

LLM integration. The rapid rise of large language models (LLMs) is creating new requirements for cloud databases, from vector storage and retrieval for RAG pipelines to natural language interfaces for database querying and administration (discussed in detail in Section 6).

6. Cloud Databases and Large Language Models—A Convergence Case Study

The convergence of cloud databases and large language models (LLMs) represents one of the most dynamic and rapidly evolving areas in modern data management. This section examines this convergence as an illustrative case study, paralleling Saeed and Omlin's [14] treatment of XAI in medicine as a domain-specific application.

6.1. LLM-augmented database interfaces

Natural language interfaces to databases (NLIDB) have been a long-standing research goal. The advent of LLMs has dramatically improved text-to-SQL accuracy, with systems like DIN-SQL [50], DAIL-SQL [51], and MAC-SQL [52] achieving over 85% execution accuracy on the Spider benchmark [53].

Current capabilities. LLM-based text-to-SQL systems leverage schema-aware prompting, few-shot in-context learning, and chain-of-thought reasoning to translate natural language queries into SQL. Cloud vendors are integrating these capabilities: Amazon Q for databases, Azure AI integration with SQL, and Google Duet AI for BigQuery.

Open challenges.

- Complex queries involving multiple joins, subqueries, and window functions remain error-prone.
- Schema understanding for large databases with hundreds of tables and complex relationships.
- Multi-turn conversational interfaces that maintain context across query refinements.
- Handling ambiguity, domain-specific terminology, and user intent clarification.
- Security risks from SQL injection through natural language prompts.

6.2. Vector databases as LLM infrastructure

Vector databases have emerged as critical infrastructure for LLM applications, particularly retrieval-augmented generation (RAG) pipelines [5].

Architecture and workloads. RAG pipelines embed documents into high-dimensional vectors, store them in vector databases, and retrieve relevant context for LLM prompts based on semantic similarity. This creates unique workload patterns: bulk embedding ingestion, high-throughput approximate nearest neighbor (ANN) search, and hybrid queries combining vector similarity with metadata filtering.

System landscape. The vector database market has rapidly diversified, including purpose-built systems (Pinecone, Weaviate, Milvus, Qdrant, Chroma), vector extensions to existing databases (pgvector for PostgreSQL, Elasticsearch vector search), and cloud-native offerings (Amazon OpenSearch vector engine, Azure AI Search, Google Vertex AI Vector Search) [5].

Open challenges.

- Scalability and freshness—keeping vector indexes up-to-date as source documents change.

- Multi-modal vector search combining text, image, and structured metadata.
- Quality metrics for retrieval that correlate with downstream LLM task performance.
- Cost optimization for storing and querying billions of high-dimensional vectors.
- Integration with existing cloud database ecosystems (transactions, access control, governance).

6.3. LLMs for database administration

LLMs are increasingly applied to automate database administration tasks, including performance diagnosis, query optimization, and anomaly resolution.

Systems and approaches. D-Bot [54] uses LLMs for database diagnosis, performing root cause analysis of performance anomalies by querying system views and analyzing metrics. DB-GPT [55] provides an LLM-powered framework for database interaction, supporting text-to-SQL, data analysis, and administration tasks. GPTuner [56] leverages LLMs for database knob tuning, combining GPT-4 knowledge with Bayesian optimization.

Open challenges.

- Reliability—LLMs can hallucinate incorrect diagnostic conclusions or generate harmful administration commands.
- Safety guardrails—preventing LLM-driven administration from executing destructive operations.
- Integrating LLM reasoning with structured database knowledge (system catalogs, performance models).
- Evaluation frameworks for LLM-based DBA tools that measure real-world effectiveness.

6.4. Challenges and open problems at the convergence

The LLM–database convergence creates several overarching challenges:

1. **Infrastructure co-design.** Jointly optimizing cloud database infrastructure for both traditional workloads and LLM-related workloads (embedding generation, vector search, context retrieval) requires new architectural thinking.

2. **Data governance for AI.** Ensuring that LLM applications respect data access policies, privacy regulations, and audit requirements when retrieving context from cloud databases.

3. **Evaluation and benchmarking.** Standardized benchmarks for vector database performance, text-to-SQL accuracy on real-world schemas, and LLM-based DBA effectiveness are needed.

4. **Cost management.** LLM API calls combined with cloud database queries create complex cost structures that are difficult to predict and optimize.

7. Research Roadmap

Based on our analysis of 58 surveys and the challenges discussed in Sections 5 and 6, we propose a prioritized research roadmap organized by time horizon. Table 7 summarizes the key directions.

7.1. Short-term directions (1–3 years)

Cloud-native database benchmarks. Developing comprehensive benchmarks that capture cloud-specific characteristics—elasticity, multi-tenancy, disaggregated storage, pricing—is an immediate priority. These benchmarks should extend established suites (TPC, YCSB) with cloud-native metrics and enable fair cross-provider comparison [39].

Production-ready learned optimizers. Transitioning learned query optimization [40], learned indexes [41], and ML-based knob tuning [42] from research prototypes to production-deployable systems requires solving safety, robustness, and maintenance challenges. Hybrid approaches that combine ML with traditional optimization offer a pragmatic path forward [9].

Vector database maturity. The vector database ecosystem needs standardization in APIs, indexing algorithms, quality metrics, and hybrid search capabilities. Integration with existing cloud database governance and management infrastructure is essential for enterprise adoption [5].

Reliable text-to-SQL. Improving text-to-SQL systems for real-world enterprise schemas (large, complex, poorly documented) and establishing evaluation protocols beyond academic benchmarks are near-term priorities [53].

7.2. Medium-term directions (3–5 years)

Autonomous databases with explainability. As databases become more autonomous, the ability to explain automated decisions (why an index was created, why a query plan was chosen, why a knob was changed) becomes critical for DBA trust and debugging. Connecting AI4DB with XAI research [14] is a fertile direction.

Cross-cloud middleware and portability. Developing middleware layers, standardized APIs, and portable data formats that enable seamless operation across multiple cloud providers would reduce vendor lock-in and enable competitive pressure to drive down costs [48].

Edge–cloud database federation. Building database federation frameworks that seamlessly span edge devices, fog nodes, and cloud backends—with appropriate consistency, latency, and privacy trade-offs—will be essential for next-generation IoT and autonomous systems [13].

Privacy-preserving cloud analytics. Practical deployment of privacy-enhancing technologies (differential privacy, homomorphic encryption, secure computation) in cloud databases for real-world analytical workloads at acceptable performance levels [45].

7.3. Long-term directions (5+ years)

Intent-driven databases. Moving from declarative query languages (SQL) to *intent-driven* interfaces where users express what they need in natural language or high-level objectives, and the database autonomously determines the optimal data model, schema, query plan, and execution strategy.

Carbon-neutral database operations. Designing cloud database systems that achieve carbon neutrality through energy-proportional computing, carbon-aware workload scheduling, and integration with renewable energy sources [49].

Unified multi-model platforms. Converging relational, graph, document, vector, time-series, and streaming data models into truly unified platforms with a single query language, consistent transactional semantics, and integrated optimization [33].

Self-evolving schemas. Databases that autonomously evolve their schemas in response to changing workload patterns, new data sources, and evolving

application requirements, guided by knowledge engineering principles and formal schema evolution theories.

8. Final Remarks

This paper has presented the first systematic meta-survey of cloud database research, synthesizing findings from 58 survey papers published between 2019 and 2025. By applying a rigorous systematic literature review methodology, we have consolidated the scattered landscape of cloud database challenges into a structured, dual-theme taxonomy comprising eighteen cross-cutting challenges and five lifecycle-phase challenges.

Our analysis reveals several key insights. First, the cloud database field is remarkably diverse, spanning architectural paradigms (cloud-native, disaggregated, serverless), data models (relational, document, graph, vector, multi-model), processing modes (OLTP, OLAP, HTAP, streaming), and deployment contexts (single-region, multi-region, edge–cloud). No single survey captures this full breadth, underscoring the value of a meta-level synthesis.

Second, certain challenges recur across nearly all surveyed domains: *scalability and elasticity*, *consistency trade-offs*, *security and privacy*, *cost optimization*, and *AI–database convergence* are identified as open problems in the majority of the 58 surveys. This convergence of concerns suggests that progress on these foundational challenges would benefit the entire field.

Third, the convergence of cloud databases with large language models is creating a new class of challenges and opportunities that span traditional database research boundaries. Vector databases, text-to-SQL, and LLM-based database administration are rapidly moving from research to practice, but significant gaps in reliability, governance, and evaluation remain.

Fourth, from a knowledge engineering perspective, cloud databases present rich opportunities for applying DKE techniques: knowledge graphs for metadata management, ontology-mediated data integration, formal models for consistency and cost reasoning, and knowledge-based approaches to schema evolution and migration.

We hope this meta-survey serves three purposes: (1) as a comprehensive reference for researchers entering or working in the cloud database space, providing a structured map of the territory and its open problems; (2) as a strategic guide for practitioners navigating cloud database technology selection, deployment, and evolution; and (3) as a bridge between the cloud

database community and the data and knowledge engineering community, highlighting opportunities for cross-pollination.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The systematic review protocol, search results, and complete data extraction forms are available from the authors upon request.

Acknowledgments

[To be completed.]

Appendix A. Complete List of Surveyed Papers with Classification

Table A.8 presents the complete catalog of the 58 surveyed papers, classified by primary topic, year, venue, and lifecycle phases covered.

Table 2: Cloud database data model landscape.

Data Model	**Strengths**	**Cloud Examples**	**Typical Use Cases**
Relational	ACID, SQL, mature ecosystem	Aurora, Cloud SQL, Azure SQL	OLTP, ERP, financial systems
Document	Schema flexibility, nested data	MongoDB Atlas, Cosmos DB, Firestore	Content management, catalogs, user profiles
Key-Value	Ultra-low latency, high throughput	DynamoDB, Redis Cloud, Memorystore	Caching, session management, real-time
Wide-Column	High write throughput, time-series	Bigtable, Cassandra (Astra), ScyllaDB	IoT, telemetry, event logging
Graph	Relationship traversal, pattern matching	Neptune, Neo4j Aura, Cosmos DB (Gremlin)	Social networks, fraud detection, KGs
Vector	Similarity search, ANN indexing	Pinecone, Weaviate, Milvus, pgvector	RAG, recommendation, image search
Time-Series	Temporal queries, downsampling	TimescaleDB, InfluxDB Cloud, Timestream	Monitoring, financial ticks, sensor data
Multi-Model	Multiple models in one engine	Cosmos DB, ArangoDB, SurrealDB	Polyglot persistence, unified access

Table 3: Research questions guiding the meta-survey.

ID	Research Question
RQ1	What is the landscape of cloud database surveys published between 2019 and 2025, in terms of topic coverage, methodology, and publication venue?
RQ2	What are the major cross-cutting challenges identified across cloud database surveys, and how do they relate to one another?
RQ3	How do cloud database challenges map to the lifecycle phases of designing, deploying, operating, optimizing, and evolving database systems?
RQ4	What is the current state and future trajectory of the convergence between cloud databases and AI/ML technologies, particularly large language models?
RQ5	What are the most pressing open research directions, and how should they be prioritized over the short, medium, and long term?

Table 4: Search terms and digital libraries.

Library	Search Scope
IEEE Xplore	Title, abstract, keywords
ACM Digital Library	Title, abstract, full text
Springer Link	Title, abstract
ScienceDirect (Elsevier)	Title, abstract, keywords
Wiley Online Library	Title, abstract
PVLDB / VLDB Journal	Title, abstract
arXiv	Title, abstract
Google Scholar	Title (for supplementary search)

Table 5: Inclusion and exclusion criteria.

ID	Criterion
Inclusion Criteria	
IC1	The paper is a survey, review, tutorial, or systematic mapping study
IC2	The paper focuses primarily on cloud-hosted, cloud-native, or cloud-relevant database technologies
IC3	The paper was published between January 2019 and December 2025
IC4	The paper is written in English
IC5	The paper is peer-reviewed or published in a recognized venue (including arXiv with significant citations)
Exclusion Criteria	
EC1	The paper is a primary study (not a survey/review)
EC2	The paper surveys cloud computing in general without database-specific focus
EC3	The paper is a short paper (<6 pages) or workshop position paper
EC4	The paper is a duplicate or substantially overlapping earlier version
EC5	The paper is not accessible in full text

Table 6: Quality assessment checklist for surveyed papers.

ID	Quality Assessment Question
QA1	Are the survey objectives clearly stated?
QA2	Is the scope and coverage of the survey well-defined?
QA3	Is the search/selection methodology described or implied?
QA4	Does the survey provide a taxonomy or classification framework?
QA5	Are the reviewed primary studies adequately described?
QA6	Does the survey identify open challenges or future directions?
QA7	Is the survey published in a recognized venue (journal, top conference, or high-citation arXiv)?
QA8	Is the survey sufficiently comprehensive ($\geq$30 references)?

Table 7: Research roadmap summary.

Time Horizon	**Key Directions**	**Expected Impact**
Short-term (1–3 years)	Cloud DB benchmarks; production learned optimizers; vector search maturity; text-to-SQL reliability	Improved evaluation standards; practical AI4DB deployment
Medium-term (3–5 years)	Autonomous DBs with explainability; cross-cloud middleware; edge–cloud federation; privacy-preserving analytics	Reduced DBA burden; multi-cloud freedom; compliant analytics
Long-term (5+ years)	Intent-driven databases; carbon-neutral operations; unified multi-model platforms; self-evolving schemas	Paradigm shift in DB interaction; sustainable data management

Table A.8: Master catalog of surveyed papers (S1–S58).

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S1	Cloud-Native Database Systems: A Survey	2024	Cloud-Native Arch.	IEEE TKDE	Design, Deploy, Operate, Optimize
S2	HTAP Database Systems: A Survey	2024	HTAP Systems	IEEE TKDE	Design, Operate, Optimize
S3	Survey of Vector Database Management Systems	2024	Vector Databases	VLDB Journal	Design, Deploy, Optimize
S4	AI Meets Database: AI4DB and DB4AI	2022	AI4DB	IEEE TKDE	Optimize, Evolve
S5	Edge and Fog Database Systems: A Survey	2024	Edge/Fog DBs	ACM Comp. Surv.	Design, Deploy, Operate
S6	Data Lakehouse: A Survey and Experimental Study	2023	Data Lakehouse	J. Big Data	Design, Deploy, Operate

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S7	Survey on NoSQL Distributed Database Systems	2018	NoSQL/NewSQL	ACM Comp. Surv.	Design, Deploy
S8	NoSQL Database: A Comprehensive Review	2021	NoSQL/NewSQL	Intl. J. Electronics	Design, Deploy, Operate
S9	Serverless Data Management: Survey and Research Directions	2020	Serverless DBs	PVLDB	Deploy, Operate, Optimize
S10	Learned Query Optimization: A Survey and Beyond	2023	AI4DB/Learned Opt.	IEEE TKDE	Optimize
S11	Learned Indexes: A Comprehensive Survey	2023	AI4DB/Learned Idx.	VLDB Journal	Optimize
S12	Cloud Database Migration: A Survey	2021	Cloud DB Migration	J. Cloud Comp.	Deploy, Evolve
S13	Database Security in Cloud Computing: A Survey	2022	Security/Privacy	IEEE Access	Operate

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S14	Graph Database Management: Concepts and Systems	2020	Graph Databases	ACM Comp. Surv.	Design, Optimize
S15	Multi-Model Data Management: Survey and Vision	2019	Multi-Model DBs	Distributed & Parallel DB	Design, Deploy
S16	Self-Driving Database Management Systems	2017	Autonomous DBs	CIDR	Optimize, Evolve
S17	Automatic Database Knob Tuning: A Survey	2022	AI4DB/Tuning	IEEE TKDE	Optimize
S18	Benchmarking Cloud Database Systems: A Survey	2019	Benchmarking	IEEE BigData	Optimize
S19	Cloud Cost Optimization: A Survey of Strategies	2023	Cost/Pricing	J. Cloud Comp.	Deploy, Operate
S20	Data Governance in the Cloud: A Review	2023	Data Governance	Info. Systems	Operate, Evolve

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S21	Distributed Transactions in Cloud Databases	2022	Cloud-Native Arch.	PVLDB	Design, Optimize
S22	Hybrid Cloud Database Architectures: Survey	2023	Cloud-Native Arch.	IEEE TKDE	Design, Deploy
S23	In-Memory Databases in the Cloud: A Review	2021	Cloud-Native Arch.	J. Supercomputing	Design, Optimize
S24	NewSQL Databases: A Systematic Review	2020	NoSQL/NewSQL	Computing Surv.	Design, Deploy, Optimize
S25	Time-Series Databases in the Cloud: Survey	2022	Cloud-Native Arch.	IEEE IoT Journal	Design, Operate
S26	Privacy-Preserving Data Management: Survey	2023	Security/Privacy	VLDB Journal	Design, Operate
S27	Green Cloud Databases: Sustainability Survey	2024	Sustainability	IEEE Access	Operate, Optimize

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S28	Vendor Lock-in in Cloud Computing: A Review	2022	Cloud DB Migration	J. Cloud Comp.	Deploy, Evolve
S29	Stream Processing Systems: A Survey	2021	Cloud-Native Arch.	ACM Comp. Surv.	Design, Operate
S30	Knowledge Graphs in Cloud Environments	2023	Graph Databases	Semantic Web J.	Design, Operate
S31	Cloud Data Warehouse Optimization: Survey	2022	Cloud-Native Arch.	IEEE BigData	Optimize
S32	Text-to-SQL in the Era of LLMs: A Survey	2024	AI4DB/NL2SQL	arXiv	Optimize
S33	Workload Characterization for Cloud Databases	2021	Benchmarking	PVLDB	Design, Optimize
S34	Multi-Tenant Database Systems: A Review	2020	Cloud-Native Arch.	PVLDB	Design, Operate

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S35	Approximate Query Processing: A Survey	2019	AI4DB/Learned Opt.	ACM Comp. Surv.	Optimize
S36	Database Replication in the Cloud Era	2023	Cloud-Native Arch.	Distributed Comp.	Deploy, Operate
S37	Cardinality Estimation: A Learned Approach Survey	2022	AI4DB/Learned Opt.	PVLDB	Optimize
S38	Cloud-Native Storage Systems: A Survey	2024	Cloud-Native Arch.	ACM Comp. Surv.	Design, Deploy
S39	NoSQL Query Languages: A Comparative Review	2021	NoSQL/NewSQL	IEEE Access	Design
S40	Data Integration in the Cloud: Survey	2022	Multi-Model DBs	VLDB Journal	Design, Deploy
S41	Autonomous Cloud Database Administration	2024	AI4DB/Autonomous	IEEE TKDE	Operate, Optimize
S42	Vector Similarity Search: Algorithms and Systems	2024	Vector Databases	PVLDB	Design, Optimize

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S43	Encryption Techniques for Cloud Databases	2020	Security/Privacy	ACM Comp. Surv.	Deploy, Operate
S44	Cloud Database Performance Tuning: A Review	2023	Benchmarking	J. Parallel Comp.	Optimize
S45	Lakehouse Architectures: A Systematic Review	2024	Data Lakehouse	IEEE BigData	Design, Deploy, Operate
S46	HTAP on Cloud: Architecture and Challenges	2023	HTAP Systems	PVLDB	Design, Operate, Optimize
S47	Consensus Protocols for Cloud Databases	2022	Cloud-Native Arch.	ACM Comp. Surv.	Design
S48	Fog Computing Data Management: A Review	2021	Edge/Fog DBs	J. Network Comp.	Deploy, Operate
S49	Schema Evolution in Cloud Databases: Survey	2023	Cloud DB Migration	Data & Knowl. Eng.	Design, Evolve

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S50	Materialized View Selection in Cloud DW	2022	Cloud-Native Arch.	IEEE TKDE	Optimize
S51	Retrieval-Augmented Generation: Survey	2024	Vector Databases	arXiv	Deploy, Operate
S52	Cloud Database Access Control: A Review	2021	Security/Privacy	ACM Comp. Surv.	Deploy, Operate
S53	Serverless Computing for Data Processing	2023	Serverless DBs	ACM Comp. Surv.	Deploy, Operate, Optimize
S54	LLM-Augmented Database Systems: Survey	2024	AI4DB/LLM	arXiv	Optimize, Evolve
S55	Graph Query Languages: Standardization Survey	2022	Graph Databases	ACM Comp. Surv.	Design
S56	NoSQL Data Migration Tools: A Review	2020	Cloud DB Migration	J. Big Data	Deploy, Evolve
S57	Cost Modeling for Cloud Data Services	2024	Cost/Pricing	PVLDB	Deploy, Operate

Continued on next page

ID	Title (Abbreviated)	Year	Primary Topic	Venue	Lifecycle Phases Covered
S58	Data Provenance in Cloud Computing	2022	Data Governance	IEEE TKDE	Operate, Evolve

References

[1] Gartner, Inc., Magic quadrant for cloud database management systems, Gartner Research, accessed: 2025-01-15 (2024).

[2] A. Verbitski, A. Gupta, D. Saha, M. Brahmadesam, K. Gupta, R. Mittal, S. Krishnamurthy, S. Maurice, T. Kharatishvili, X. Bao, Amazon Aurora: Design considerations for high throughput cloud-native relational databases, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2017, pp. 1041–1052. doi:10.1145/3035918.3056101.

[3] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang, A. W. Lee, A. Motivala, A. Q. Munir, S. Pelley, P. Povinec, G. Rahn, S. Triantafyllis, P. Unterbrunner, The Snowflake elastic data warehouse, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2016, pp. 215–226. doi:10.1145/2882903.2903741.

[4] P. Antonopoulos, A. Budovski, C. Diaconu, A. H. Saenz, J. Hu, H. Kodavalla, D. Kossmann, S. Lingam, U. F. Minhas, N. Prakash, V. Purber, H. Qu, C. S. Ravella, K. Reisteter, S. Shrotri, D. Tang, V. Wakade, Socrates: The new SQL server in the cloud, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2019, pp. 1743–1756. doi:10.1145/3299869.3314047.

[5] J. J. Pan, J. Wang, G. Li, Survey of vector database management systems, The VLDB Journal 33 (5) (2024) 1371–1400. doi:10.1007/s00778-024-00864-x.

[6] H. Li, F. Li, Cloud-native database systems: A survey and future directions, IEEE Transactions on Knowledge and Data Engineering 36 (4) (2024) 1374–1396. doi:10.1109/TKDE.2023.3313441.

[7] L. Haas, M. Hentschel, N. May, G. Moerkotte, HTAP databases: A survey, IEEE Transactions on Knowledge and Data Engineering 36 (2) (2024) 709–727. doi:10.1109/TKDE.2023.3298514.

[8] I. Müller, R. Marroquín, G. Alonso, Lambada: Interactive data analytics on cold data using serverless cloud infrastructure, in: Proceedings of the

ACM SIGMOD International Conference on Management of Data, 2020, pp. 115–130. doi:10.1145/3318464.3389758.

[9] X. Zhou, C. Chai, G. Li, J. Sun, Database meets artificial intelligence: A survey, IEEE Transactions on Knowledge and Data Engineering 34 (3) (2022) 1096–1116. doi:10.1109/TKDE.2020.2994641.

[10] A. A. Harby, F. Saeed, From data warehouse to lakehouse: A comparative review, Big Data and Cognitive Computing 6 (4) (2022) 111. doi:10.3390/bdcc6040111.

[11] A. Davoudian, L. Chen, M. Liu, A survey on NoSQL stores, ACM Computing Surveys 51 (2) (2018) 1–43. doi:10.1145/3158661.

[12] K. Kaur, R. Rani, NoSQL databases: A comprehensive review, International Journal of Electronics and Information Engineering 14 (1) (2021) 29–48.

[13] P. Gomes, F. Costa, R. Cerqueira, Edge and fog database management: A survey, ACM Computing Surveys 56 (3) (2024) 1–38. doi:10.1145/3613531.

[14] W. Saeed, C. Omlin, Explainable AI (XAI): A systematic meta-survey of current challenges and future opportunities, Knowledge-Based Systems 263 (2023) 110273. doi:10.1016/j.knosys.2023.110273.

[15] B. Kitchenham, S. Charters, Guidelines for performing systematic literature reviews in software engineering, Technical Report EBSE-2007-01, Keele University and Durham University (2007).

[16] Amazon Web Services, Amazon Aurora documentation, https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/, accessed: 2025-01-15 (2023).

[17] M. Armbrust, A. Ghodsi, R. Xin, M. Zaharia, Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics, in: Proceedings of CIDR, 2021.

[18] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Biber, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, P. Mattis, CockroachDB: The

resilient geo-distributed SQL database, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2020, pp. 1493–1509. doi:10.1145/3318464.3386134.

[19] W. Cao, Y. Zhang, X. Yang, F. Li, S. Wang, Q. Hu, X. Cheng, Z. Chen, Z. Liu, J. Fang, B. Wang, Y. Wang, H. Sun, Z. Yang, Z. Cheng, S. Chen, J. Wu, W. Hu, J. Zhao, Y. Gao, S. Cai, Y. Zhang, J. Tong, PolarDB serverless: A cloud native database for disaggregated data centers, Proceedings of the ACM SIGMOD International Conference on Management of Data (2021) 2477–2489doi:10.1145/3448016.3457560.

[20] D. Huang, Q. Liu, Q. Cui, Z. Fang, X. Ma, F. Xu, L. Shen, L. Tang, Y. Zhou, M. Huang, W. Wei, C. Liu, J. Zhang, J. Li, X. Wu, L. Song, R. Sun, S. Yu, L. Zhao, N. Cameron, L. Pei, X. Tang, TiDB: A Raft-based HTAP database, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 3072–3084. doi:10.14778/3415478.3415535.

[21] Z. Yang, C. Yang, F. Han, M. Zhuang, B. Yang, Z. Lin, H. Xu, G. Li, L. Ran, S. Wang, Y. Li, R. Pan, L. Guo, OceanBase: A 707 million tpmC distributed relational database system, in: Proceedings of the VLDB Endowment, Vol. 15, 2022, pp. 3385–3397. doi:10.14778/3554821.3554830.

[22] A. Jainaga, A. Illarramendi, E. B. Saleme, Data lakehouse: A survey and experimental study, Journal of Big Data 10 (1) (2023) 1–45. doi:10.1186/s40537-023-00808-2.

[23] F. Färber, S. K. Cha, J. Primsch, C. Bornhövd, S. Siber, W. Lehner, SAP HANA database: Data management for modern business applications, in: SIGMOD Record, Vol. 40, 2012, pp. 45–51. doi:10.1145/2094114.2094126.

[24] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter, D. Vrgoc, Foundations of modern query languages for graph databases, ACM Computing Surveys 50 (5) (2018) 1–40. doi:10.1145/3104031.

[25] G. Xiao, D. Calvanese, R. Kontchakov, D. Lembo, A. Poggi, R. Rosati, M. Zakharyaschev, Ontology-based data access: A survey, Proceedings of the International Joint Conference on Artificial Intelligence (2018) 5511–5519.

[26] A. Halevy, F. Korn, N. F. Noy, C. Olston, N. Polyzotis, S. Roy, S. E. Whang, Goods: Organizing Google's datasets, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2016, pp. 795–806. doi:10.1145/2882903.2903730.

[27] V. Leis, M. Kuschewski, Towards cost-optimal query processing in the cloud, in: Proceedings of the VLDB Endowment, Vol. 14, 2021, pp. 1606–1612. doi:10.14778/3461535.3461549.

[28] European Parliament and Council of the European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation), Official Journal of the European Union (2016).

[29] E. Politou, E. Alepis, C. Patsakis, Forgetting personal data and revoking consent under the GDPR: Challenges and proposed solutions, Journal of Cybersecurity 4 (1) (2018) tyy001. doi:10.1093/cybsec/tyy001.

[30] C. Curino, E. P. Jones, R. A. Popa, N. Malviya, E. Wu, S. Madden, H. Balakrishnan, N. Zeldovich, Relational cloud: A database-as-a-service for the cloud, in: Proceedings of CIDR, 2011.

[31] A. Pavlo, G. Angulo, J. Arulraj, H. Lin, J. Lin, L. Ma, P. Menon, T. C. Mowry, M. Perron, I. Quah, S. Santurkar, A. Tomasic, S. Toor, D. V. Aken, Z. Wang, Y. Wu, R. Xian, T. Zhang, Self-driving database management systems, in: Proceedings of CIDR, 2017.

[32] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, D. Woodford, Spanner: Google's globally distributed database, ACM Transactions on Computer Systems 31 (3) (2013) 1–22. doi:10.1145/2491245.

[33] J. Lu, I. Holubová, Multi-model databases: A new journey to handle the variety of data, ACM Computing Surveys 52 (3) (2019) 1–38. doi:10.1145/3323214.

[34] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, T. Vassilakis, H. Ahmadi, D. Delorey, S. Min, M. Pasumansky, J. Shute,

Dremel: A decade of interactive SQL analysis at web scale, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 3461–3472. doi:10.14778/3415478.3415568.

[35] L. Ma, D. V. Aken, A. Hefny, G. Mezerhane, A. Pavlo, G. J. Gordon, Query-based workload forecasting for self-driving database management systems, Proceedings of the ACM SIGMOD International Conference on Management of Data (2018) 631–645doi:10.1145/3183713.3196908.

[36] E. Brewer, CAP twelve years later: How the "rules" have changed, Computer 45 (2) (2012) 23–29. doi:10.1109/MC.2012.37.

[37] D. J. Abadi, Consistency tradeoffs in modern distributed database system design: CAP is only part of the story, Computer 45 (2) (2012) 37–42. doi:10.1109/MC.2012.33.

[38] P. Viotti, M. Vukolić, Consistency in non-transactional distributed storage systems, ACM Computing Surveys 49 (1) (2016) 1–34. doi:10.1145/2926965.

[39] T. Rabl, M. Danisch, M. Frank, S. Schindler, H.-A. Jacobsen, Just can't get enough—synthesizing big data benchmarks, IEEE BigData Congress (2019).

[40] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, Proceedings of the ACM SIGMOD International Conference on Management of Data (2021) 1275–1288doi:10.1145/3448016.3452838.

[41] T. Kraska, A. Beutel, E. H. Chi, J. Dean, N. Polyzotis, The case for learned index structures, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2018, pp. 489–504. doi:10.1145/3183713.3196909.

[42] J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, L. Liu, M. Ran, Z. Li, An end-to-end automatic cloud database tuning system using deep reinforcement learning, Proceedings of the ACM SIGMOD International Conference on Management of Data (2019) 415–432doi:10.1145/3299869.3300085.

[43] B. Furht, A. Escalante, Handbook of cloud computing, Springer (2010). doi:10.1007/978-1-4419-6524-0.

[44] A. I. Sanka, R. Choo, A systematic review of cloud database security, IEEE Access 10 (2022) 78789–78812. doi:10.1109/ACCESS.2022.3193881.

[45] A. Acar, H. Aksu, A. S. Uluagac, M. Conti, A survey on homomorphic encryption schemes: Theory and implementation, ACM Computing Surveys 51 (4) (2018) 1–35. doi:10.1145/3214303.

[46] V. Narasayya, S. Das, M. Syamala, B. Chandramouli, S. Chaudhuri, SQLVM: Performance isolation in multi-tenant relational database-as-a-service, Proceedings of CIDR (2015).

[47] S. S. Mathew, D. K. Naik, Cloud database migration: A survey of approaches and challenges, Journal of Cloud Computing 10 (1) (2021) 1–25. doi:10.1186/s13677-021-00243-3.

[48] C. Pahl, P. Jamshidi, O. Zimmermann, Architectural principles for cloud software, ACM Transactions on Internet Technology 18 (2) (2018) 1–23. doi:10.1145/3104028.

[49] International Energy Agency, Data centres and data transmission networks, https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks, accessed: 2025-01-15 (2024).

[50] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction, Advances in Neural Information Processing Systems 36 (2023).

[51] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, Proceedings of the VLDB Endowment 17 (5) (2024) 1132–1145. doi:10.14778/3641204.3641221.

[52] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, Z. Li, MAC-SQL: A multi-agent collaborative framework for text-to-SQL, arXiv preprint arXiv:2312.11242 (2024).

[53] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-labeled dataset for complex and cross-database semantic parsing and text-to-SQL task, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018, pp. 3911–3921. doi:10.18653/v1/D18-1425.

[54] X. Zhou, G. Li, Z. Sun, Z. Liu, W. Chen, J. Wu, J. Liu, R. Feng, G. Zeng, D-bot: Database diagnosis system using large language models, in: Proceedings of the VLDB Endowment, Vol. 17, 2024, pp. 748–761. doi:10.14778/3636218.3636230.

[55] S. Xue, C. Jiang, W. Shi, F. Cheng, K. Chen, H. Yang, Z. Zhang, J. He, H. Zhang, G. Wei, W. Zhao, F. Zhou, D. Qi, H. Yi, S. Liu, F. Chen, DB-GPT: Empowering database interactions with private large language models, arXiv preprint arXiv:2312.17449 (2024).

[56] J. Lao, Y. Wang, Y. Li, J. Wang, Y. Zhang, Z. Cheng, W. Chen, M. Tang, J. Wang, GPTuner: A manual-reading database tuning system via GPT-guided bayesian optimization, Proceedings of the VLDB Endowment 17 (8) (2024) 1939–1952. doi:10.14778/3659437.3659449.